

SW Engineering CSC648-848 Fall 2025

GatorsMatch Tutors

Team 01

Name	Role(s)
Amer Music - amusic@mail.sfsu.edu	Project Lead & Frontend Developer
Arrie Sekhon	Frontend Lead
Derek Ye	Backend Lead
Rishika Bharodiya	GitHub Master
Stanley Wang	Backend Support
Alonzo Manosca	GitHub Support
Daniel Castillo	Frontend Support

Document History

	Submitted Date	Revised Date
Milestone 5	2025/12/14	
URL	http://18.191.185.168/	

Index

[1\) Product Summary](#)

[2\) Milestone Documents \(M1-M4\)](#)

[3\) Product Screenshots](#)

[4\) Database Organization](#)

[5\) GitHub organization](#)

[6\) Google analytics Stats Plot for Your WWW Site](#)

[7\) Project Management](#)

[8\) Team Member Self-assessment and Contributions](#)

1.) Product Summary

Product Name: GatorMatch Tutors

Product Description (Plain English, 1 paragraph)

GatorMatch Tutors is a safe, simple, SFSU-only platform that helps students quickly find credible peer tutors for the exact courses they're taking. Any @sfsu.edu student can search by course code (e.g., CSC 340), browse approved tutor listings, check availability, and send a one-round in-site message to connect—no outside email or messaging apps required. Tutor profiles highlight meaningful trust signals such as course expertise, availability, campus meeting options, and SFSU-verified eligibility. Tutors can easily create and update listings, and every listing goes through an Admin approval step to ensure accuracy and safety. What makes GatorMatch unique is its SFSU-specific design: course-centric search, campus location tags, semester-aware availability, and filters that never show empty results. The platform provides a moderated, campus-focused directory that is easy for students, tutors, administrators, and faculty to rely on.

Student Functions

1. Students can register using an @sfsu.edu email address.
2. Students can sign in with an @sfsu.edu email address.
3. Students can browse and search for tutors using data-driven filters that hide options with zero matches.
4. Students can view full tutor listings including bio, subjects, courses, availability, and location tags.
5. Students can send a one-round in-site "Contact Tutor" message (no replies allowed).

Tutor Functions

6. Students can enable a Tutor role to create tutor listings.
7. Tutors can create or edit their listings, including headline, bio, subjects, course prefixes, availability, and location tags.
8. Tutors can submit listings for Admin approval and see their status (Draft, Pending, Approved, Rejected).

Admin Functions

9. Admins can approve or reject listings and provide rejection reasons.
10. Admins can access a role-based dashboard with an approval queue and moderation tools.

System-Level Features

11. Course-centric and SFSU-specific discovery (course filters, campus locations, semester-friendly availability).
12. Role-based dashboards for Students, Tutors, and Admins.

2.) Milestone Documents (M1-M4):

Milestone 1

1.) Executive Summary

GatorMatch Tutors is an SFSU-only web application that makes it fast, safe, and credible for students to find peer tutors for the exact courses they're taking. Instead of bouncing across generic marketplaces and informal channels, any @sfsu.edu student can search by course code (e.g., CSC 340), see meeting options at a glance (Zoom or on-campus), review concise tutor profiles, and send a single in-site message to start contact—no email chase or chat threads. Every tutor profile clearly surfaces trust signals that matter to students and faculty: a 5-star rating generated by verified students who have used that tutor's services, the total hours tutored to date, and an eligibility badge confirming the tutor has completed the course with a minimum grade of 90% and/or holds a professor's recommendation. Tutors can create and manage listings with clear status (Draft, Pending, Approved), and every listing is reviewed by an administrator who can approve, reject with a reason, or later unpublish to keep the catalog clean and trustworthy.

What makes GatorMatch distinctive is its SFSU fit. Discovery is course-centric and campus-aware: listings are tagged with official course codes and campus locations such as the J. Paul Leonard Library, and availability is expressed in semester-friendly terms students actually use ("post-lab hours," "midterm week evenings"). Filters are data-driven so options with zero matches never waste a student's time. The result is a moderated, auditable directory that shortens time-to-help for students, channels relevant inquiries to tutors, and provides the university a safe, focused service without payments or external messaging.

2.) Personas

Persona 1 — Sophia Nguyen

	About: Jordan reviews new tutor listings, checks course accuracy and appropriateness, and keeps the directory clean. Time is scarce; decisions must be quick and auditable.
---	---

Role: Student Seeker (Primary) | Age: 20 | Location: Daly City ↔ SFSU
(commuter) | Status: B.S. CS, 3rd year

“I just need the right tutor for CSC 340 and a quick way to contact them—no endless back-and-forth.”

 Motivations:

- Find a tutor by course prefix + number in minutes
- See availability and meeting mode up front
- Keep everything inside the site (no email chase)

 Goals:

- Results that match *CSC 340* with live counts on filters
- Scan tutor cards with course tags and Library/Zoom badges
- Send a single in-site message and get a clear confirmation

 Frustrations:

- Irrelevant results when filtering by course
- Filters that show options with 0 matches
- Long forms or multi-step chat/DM outside the site

 Demographics / Context:

- Devices: 13" laptop; iPhone on the go
- Study window: late evenings; variable Wi-Fi
- Accessibility: prefers high contrast, large tap targets

 Primary Scenario (What, not How)

Before midterms, Sophia searches for “CSC 340,” filters results to Zoom, skims a few relevant tutor profiles, opens one that looks best, and sends a quick message with her request — all within the site.

Persona 2 — Marco Alvarez

	About: Marco mentor’s undergraduates in CSC 340 and CSC 648. He values a simple, efficient way to post listings with clear approval feedback. He wants inquiries that are relevant to his courses—no spam or long email threads.
---	--

Role: Tutor (Provider) | Age: 24 | Location: San Francisco | Status: M.S. CS, 1st year
“Make it easy to publish a legit listing and let students contact me once with the right context.”

 Motivations:

- Quick Tutor enablement after sign-up
- Clean form for headline, bio, course prefixes, availability, Zoom/campus tags
- Visibility in course-centric searches

 Goals:

- Listing Submitted → Pending → Approved with reason if rejected
- Appear when students search by course code
- Receive a single, contextual in-site message

 Frustrations:

- Ambiguous forms and unclear approval outcomes
- Off-topic inquiries (“Do you teach Math?” for a CSC listing)
- Being forced into email chains or outside chats

 Demographics / Context:

- Devices: Laptop to edit, phone to check messages
- Time: 20-minute gaps between classes
- Privacy: prefers in-site contact over personal email

 Primary Scenario (What, not How):

After class, Marco enables Tutor role, adds *CSC 340/648*, Library & Zoom tags, sets availability, submits; later sees Approved and starts receiving targeted, one-round messages.

Persona 3 — Jordan Lee

	About: Jordan reviews new tutor listings, checks course accuracy and appropriateness, and keeps the directory clean. Time is scarce; decisions must be quick and auditable.
---	---

Role: Administrator (Moderator) | Age: 28–35 | Location: SFSU | Status: Staff / Appointed Moderator

“I need to triage listings fast with enough context to approve or reject—and maintain a clean catalog.”

Motivations:

- Approval Queue with compact snapshots (headline, bio, course tags, meeting mode)
- Approve / Reject with reason, and Unpublish/Delete later
- Basic moderation on users/content when needed

Goals:

- Process new listings within 24–48 hours
- Maintain audit trail (who/when/why)
- Keep catalog SFSU-appropriate and course-accurate

Frustrations:

- Sparse listings lacking course/meeting tags
- No reason field or history of actions
- Needing multiple screens to decide

Demographics / Context:

- Devices: Laptop; short review windows between duties
- Interaction style: bulk triage, keyboard driven
- Signals: clear status badges; queues/filters

Primary Scenario (What, not How) :

Jordan opens Approval Queue, scans listing snapshots, Approves good entries, Rejects with reason where needed, and later Unpublishes any reported listing.

3.) High-Level Uses Cases

Course-Centric Tutor Discovery

Actors: Sophia (Student Seeker)

Before midterms, Sophia wants help for CSC 340. She enters the course code, limits results to meeting modes that work for her (Zoom or on-campus), and quickly scans only relevant tutor cards that clearly show course tags, current availability, and a high average star-rating. She opens a profile she likes, confirms it matches her schedule and mode, and proceeds to contact that single tutor without needing external email or multi-step chats. If no good match appears, she adjusts filters that only show options with real matches, so she doesn't waste time.

Publish a Tutor Listing

Actors: Marco (Tutor)

Between classes, Marco decides to offer tutoring for CSC 340 and CSC 648. He enables the Tutor role by creating a **tutor profile** with his SFSU email and completes a concise listing that includes a headline, a short bio, where he can meet (Zoom or on-campus), and 1-hour availability slots. He adds only courses he has completed with a grade of $\geq 90\%$ and uploads up to three verification documents (e.g., transcript screenshots) that match those courses. After certifying the policies, he submits the listing and watches the status progress from Submitted to Under Review to Approved (or Changes Requested). Once approved, his listing becomes discoverable in course-centric searches so inquiries he receives are on-topic and include useful course context.

Moderate Listings and Keep the Catalog Clean

Actors: Jordan (Administrator)

During a short review window, Jordan opens the approval queue and sorts any new tutor submissions. For each, Jordan checks that course tags, bio, and meeting modes are appropriate for SFSU. Good entries are approved; unclear or off-scope entries are rejected with a brief reason, so tutors know how to improve. Later, if a listing is reported, Jordan can unpublish or remove it and maintain an auditable record of actions for catalog transparency and ensure that it is SFSU-appropriate. Unclear or off-scope entries are rejected by selecting predefined reason(s) from a multi-select list and, when needed, adding a short note so tutors know exactly how to improve.

One-Round In-Site Message with Clear Confirmation

Actors: Sophia (Student Seeker), Marco (Tutor)

After opening Marco's profile, Sophia sends one concise, in-site **message** that includes course, topic focus, and preferred meeting mode. She immediately sees that the **message** was sent; there's no expectation of a threaded chat. Marco reviews the **message** in his inbox during a break and has the necessary context to accept or propose a time through his chosen channel or meeting mode outside the system. Both can later reference the single **message** in their role-based dashboards (Sent/Inbox) without exposing personal emails.

SFSU-Specific Discovery: Semester-Aware Availability & Campus Location Match (*differentiator*)

Actors: Sophia (Student Seeker), Marco (Tutor)

Sophia needs someone available around SFSU's semester rhythm and prefers to meet at the J. Paul Leonard Library. She filters by course code and selects campus location tags to see tutors who explicitly meet at SFSU spaces. Availability is expressed in semester-aware terms students recognize (e.g., "post-lab hours," "midterm week evenings"), so she can align help with real campus schedules. Semester-aware phrases are saved as lightweight tags per course/section and automatically resolved into real time windows using the term and course calendars (labs, lectures, midterm weeks). At search time, these windows are intersected with the tutor's existing hourly slots and location tags, keeping results accurate without changing the current calendar UI. This SFSU-specific combination of course-centric search, campus location tags, and semester-aware availability guides her to a fit she couldn't easily find on generic tutoring sites.

4.) List of Main Data items and Entities – Data Glossary/Description

- Key Users

- Unregistered Student

Definition	Any user who visits the platform without logging in
Permissions	Can browse and search tutor listings, but cannot send messages, create or edit listings, or access dashboards
Usage	Enables casual browsing before sign-up, encouraging new users to register once they find relevant tutors

- Registered Student

Definition	A verified SFSU student logged into the system to find tutoring help
Permissions	Can browse/search tutor listings and send a one-time message to tutors
Usage	Primary consumer of tutoring services; uses the platform to find and contact peer tutors

- Registered Tutor

Definition	A verified SFSU student providing tutoring services through the platform
Permissions	Can create and update tutor listings; submit listings for admin approval
Usage	Primary provider of tutoring services; maintains visibility and

	availability through listings
--	-------------------------------

○ Admin

Definition	A user responsible for managing and moderating the platform
Permissions	Can view all tutor listings, approve or reject posts (with reasons), unpublish or delete content, and moderate users
Usage	Ensures platform quality, verifies listings, and maintains a safe environment for all users

● Data Items

○ User Profile

Definition	Stores verified SFSU user information and determines the user's role (Student, Tutor, or Admin)
Sub Items	• Full Name • SFSU Email (@sfsu.edu) • Role (Student, Tutor, Admin) • Profile Picture (optional) • Registration Date • Account Status (Active / Blocked)
Input	Created during registration and updated by the user
Usage	Used for authentication, access control, and displaying role-based dashboards

○ Tutor Profile

Definition	A profile-like listing created by a tutor that advertises their services
Sub Items	• Tutor Name • Subject Expertise (e.g., CSC 648, MATH 226) • Availability (day and time) • Campus Location Tag (Library, Zoom, etc.) • Additional Notes / Description • Status (Pending, Approved, Rejected) • Admin Review Reason (optional)
Input	Created and submitted by Tutor; requires Admin approval before being public
Usage	Displayed in searchable listings for Students; visibility controlled by approval status

○ Message

Definition	A one-round, in-site message sent from a Student to a Tutor regarding a specific listing
Sub Items	• Message ID • Sender (Student) • Receiver (Tutor) • Linked Listing ID • Message Content • Timestamp • Status (Sent, Read)
Input	Created automatically when a Student sends a one-round message to a Tutor
Usage	Appears in the Student's "Sent" box and Tutor's "Inbox"; no replies or external email allowed

○ Admin Review Log

Definition	A record of all Admin actions taken on Tutor Posts or Messages for accountability and moderation
Sub Items	• Admin ID • Action Type (Approve, Reject, Unpublish, Delete) • Target Post / Message ID • Timestamp • Reason / Notes
Input	Automatically generated when Admin performs an action
Usage	Enables transparent moderation tracking and supports audit or rollback if needed

5.) High-Level Functional Requirements

- **FR-1 SFSU Registration & Sign-in:** Users must register/sign in with @sfsu.edu (suffix check only).
- **FR-2 Tutor Role Enablement:** Students can enable a Tutor role to create listings.
- **FR-3 Create/Update Listing:** Tutor can add/edit headline, bio, subjects, course prefixes, availability, location tags, optional display-only rate.
- **FR-4 Admin Approval & Moderation:** Admin can approve/reject (with reason), unpublish, delete listings; moderate users/content.
- **FR-5 Browse & Search:** Search and filter by Term, Session, Subject, Course Number, Location; options are data-driven and disabled/hidden when there are 0 matches.

- **FR-6 Listing Detail:** Detail page shows bio, subjects, course prefixes, availability, location tags, and Contact Tutor.
- **FR-7 One-Round In-Site Messaging:** Student can send one message to a tutor; no replies; no email clients.
- **FR-8 Role-Based Dashboards:** Student (Sent, profile); Tutor (Listings, Inbox, profile); Admin (Approval Queue, Moderation).
- **FR-9 SFSU-Specific Discovery:** Course-centric filters, campus location tags, and semester-aware availability text.
- **FR-10 Analytics & Banner:** Google Analytics enabled; exact required banner text on every page.

6.) Non-Functional Requirements (NFR)

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in M0
2. Application shall be optimized for standard desktop/laptop browsers e.g must render correctly the two latest versions of two major browsers.
3. All or selected application functions shall be rendered well on mobile devices (no native app to be developed)
4. Posting of tutor information and messaging to tutors shall be limited only to SFSU students
5. Critical data shall be stored in the database on the team;s deployment server.
6. No more than 50 concurrent users shall be accessing the application at any time.
7. Privacy of users shall be protected.
8. The language used shall be English (no localization needed)
9. Application shall be very easy to use and intuitive.
10. Application shall follow established architecture patterns.
11. Application code and its repository shall be easy to inspect and maintain.
12. Google analytics shall be used.
13. No e-mail clients shall be allowed. Interested users (clients) can only message service providers via in-site messaging. One round of messaging (from client to service provider) is enough for this application. No chat functions shall be developed or integrated.

14. Pay functionality (e.g. paying for goods and services) shall not be implemented nor simulated in UI.
15. Site security: basic best practices shall be applied (as covered in the class) for main data items.
16. Media formats shall be standard as used in the market today.
17. Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools.
18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages *“SFSU Software Engineering Project CSC 648-848, Fall 2025. For Demonstration Only”* at the top of the WWW page Nav bar. (Important so as to not confuse this with a real application).

7.) Competitive Analysis

Feature	Khan Academy	Wyzant	Preply	Skooli	Our site
Tutor profiles	-	+	+	+	+
Course searching	+	+	+	+	++
Schedule tutoring session	-	+	+	+	++
Dark mode	-	-	-	-	+
Tutor messaging	-	+	+	+	++

For our tutoring website, we plan on improving on the course search, tutor scheduling, and the tutor messaging systems that more popular tutoring websites have. We want to improve on the search feature by adding a student rating next to a certain tutor or course. Sort of like a built-in RateMyProfessor, but for tutors! To improve on the scheduling system that the popular websites have, we would like to add a feature that lets students put in their schedule and then our website picks the best times for both the student and the tutor. To keep communication simple and controlled, students may send one message to a tutor outside of a

tutoring session; tutors cannot reply in the system, and students cannot send follow-ups. Any discussion occurs only during scheduled sessions. Email is not used for messaging: the platform does not send or receive emails, forward messages to email, or expose email addresses. Messages are viewable to the recipient tutor in-app only, and if a response is required, tutors will address it during the next session (or the student can book a session).

As a little bonus we want to add a native dark mode to our website that can be toggled on and off!

8.) High-Level System Architecture and Technologies Used

1. List all main software components and versions (DB, WWW server)

- **Operating System:** Ubuntu 22.04 LTS
- **Web Server:** NGINX 1.18.0 + Gunicorn 23.0.0
- **Database:** MySQL 8.0.37
- **Programming Language:** Python 3.10.18
- **Framework:** Flask 3.2.1 (used to build the web application) + Jinja2 3.1.6 (template engine)
- **Version Control:** Git + GitHub
- **Dependency Management:** requirements.txt file + pip 25.2
- **Front End:** HTML5 + CSS3 (Custom styles), Vanilla JS (for interactions)

2. List deployment cloud service you plan to use

We are deploying our web application on **Amazon Web Services (AWS)** using an **EC2 t3.micro** instance.

3. List front-end frameworks you will use

The site uses Flask-rendered HTML templates (Jinja templates) with custom CSS, no separate front-end framework.

4. List browsers you plan to support (choose 2 market leading browsers, last two versions from each)

We plan to support the following browsers:

- **Google Chrome** – 142 & 141
- **Safari** – 26.0.1 & 26.0

This will make sure our tutoring website works well for most users on both Windows and macOS.

5. List any major additional external open-source APIs you plan to use

We plan to use the following APIs in our project:

- **Google Analytics** – to track website visitors and understand user activity.
- **Zoom API** – to create and manage tutoring session links for students and tutors.
- **Google Maps API** – to show tutoring locations or directions (for future updates).

9.) Use of Gen AI Tools (ChatGPT, Copilot, etc.)

Our team didn't really rely on any GenAI tools for content generation for this milestone. Most of the work done for this milestone involved team discussion and individual writing. However we could see how these tools could be helpful for other teams when it comes to brainstorming personas, use case narratives or generating some ways to format certain sections. For future milestones we might explore GenAI tools a little more once we start developing and documenting more code.

10.) Team and Roles

Name	Email	Role
Amer Music	amusic@mail.sfsu.edu	Team Lead
Arric Sekhon	asekhon@sfsu.edu	Frontend Lead
Derek Ye	dye3@sfsu.edu	Backend Lead
Rishika Bharodiya	rbharodiya@sfsu.edu	Github Master
Stanley Wang	hwang26@sfsu.edu	Backend Support
Alonzo Manosca	amanosca1@sfsu.edu	Github Support
Daniel Castillo	dcastillo8@sfsu.edu	Frontend Support

11.) Team Lead Checklist

Checklist Item	Status	Comments
So far, all team members are fully engaged and attending team sessions when required	On Track (on-going)	Consistently engaged via discord & attending classes.
Team found a time slot to meet outside of the class	Done	Fridays at 5:30 pm
Team ready and able to use the chosen back and front-end frameworks and those who need to learn are working on learning and practicing	On Track (on-going)	
Team reviewed class slides on requirements and use cases before drafting Milestone 1	Done	
Team reviewed non-functional requirements from “How to start...” document and developed Milestone 1 consistently	Done	
Team lead checked Milestone 1 document for quality, completeness, formatting and compliance with instructions before the submission	On Track (on-going)	
Team lead ensured that all team members read the final M1 and agree/understand it before submission	Done	
Team shared and discussed experience with GenAI tools among themselves	On Track	
Github organized as discussed in class (e.g. master branch, development branch, folder for milestone documents etc.)	On Track (on-going)	

Milestone 2

1.) Executive Summary

GatorMatch Tutors is an SFSU-only web application that makes it fast, safe, and credible for students to find peer tutors for the exact courses they’re taking. Instead of bouncing across generic marketplaces and informal channels, any @sfsu.edu student can search by course code (e.g., CSC 340), see meeting options

at a glance (Zoom or on-campus), review concise tutor profiles, and send a single in-site message to start contact—no email chase or chat threads. Every tutor profile clearly surfaces trust signals that matter to students and faculty: a 5-star rating generated by verified students who have used that tutor’s services, the total hours tutored to date, and an eligibility badge confirming the tutor has completed the course with a minimum grade of 90% and/or holds a professor’s recommendation. Tutors can create and manage listings with clear status (Draft, Pending, Approved), and every listing is reviewed by an administrator who can approve, reject with a reason, or later unpublish to keep the catalog clean and trustworthy.

What makes GatorMatch distinctive is its SFSU fit. Discovery is course-centric and campus-aware: listings are tagged with official course codes and campus locations such as the J. Paul Leonard Library, and availability is expressed in semester-friendly terms students actually use (“post-lab hours,” “midterm week evenings”). Filters are data-driven so options with zero matches never waste a student’s time. The result is a moderated, auditable directory that shortens time-to-help for students, channels relevant inquiries to tutors, and provides the university a safe, focused service without payments or external messaging.

2.) List of All Data Items & Entities

- Key Users
 - Unregistered Student

Definition	Any user who visits the platform without logging in
------------	---

Permissions	Can browse and search tutor listings, but cannot send messages, create or edit listings, or access dashboards
Usage	Enables casual browsing before sign-up, encouraging new users to register once they find relevant tutors

○ Registered Student

Definition	A verified SFSU student logged into the system to find tutoring help
Permissions	Can browse/search tutor listings and send a one-time message to tutors
Usage	Primary consumer of tutoring services; uses the platform to find and contact peer tutors

○ Registered Tutor

Definition	A verified SFSU student providing tutoring services through the platform
Permissions	Can create and update tutor listings; submit listings for admin approval
Usage	Primary provider of tutoring services; maintains visibility and availability through listings

○ Admin

Definition	A user responsible for managing and moderating the platform
Permissions	Can view all tutor listings, approve or reject posts (with reasons), unpublish or delete content, and moderate users
Usage	Ensures platform quality, verifies listings, and maintains a safe environment for all users

● Data Items

○ User Profile

Definition	Stores verified SFSU user information and determines the user's role (Student, Tutor, or Admin)
Sub Items	• Full Name • SFSU Email (@sfsu.edu) • Role (Student, Tutor, Admin) • Profile Picture (optional)

	• Registration Date • Account Status (Active / Blocked)
Input	Created during registration and updated by the user
Usage	Used for authentication, access control, and displaying role-based dashboards

○ Tutor Profile

Definition	A profile-like listing created by a tutor that advertises their services
Sub Items	• Tutor Name • Subject Expertise (e.g., CSC 648, MATH 226) • Availability (day and time) • Campus Location Tag (Library, Zoom, etc.) • Additional Notes / Description • Status (Pending, Approved, Rejected) • Admin Review Reason (optional)
Input	Created and submitted by Tutor; requires Admin approval before being public
Usage	Displayed in searchable listings for Students; visibility controlled by approval status

○ Message

Definition	A one-round, in-site message sent from a Student to a Tutor regarding a specific listing
Sub Items	• Message ID • Sender (Student) • Receiver (Tutor) • Linked Listing ID • Message Content • Timestamp • Status (Sent, Read)
Input	Created automatically when a Student sends a one-round message to a Tutor
Usage	Appears in the Student's "Sent" box and Tutor's "Inbox"; no replies or external email allowed

○ Admin Review Log

Definition	A record of all Admin actions taken on Tutor Posts or Messages for accountability and moderation
Sub Items	• Admin ID

	• Action Type (Approve, Reject, Unpublish, Delete) • Target Post / Message ID • Timestamp • Reason / Notes
Input	Automatically generated when Admin performs an action
Usage	Enables transparent moderation tracking and supports audit or rollback if needed

○ Search Tutors

Definition	Store search preferences from students to find matching tutors.
Sub Items	• Course Name • Keyword • Subject • Tutor's Profile • Tutor's Rating
Input	From the student's search form.
Usage	Filters Tutor data (profile, availability, reviews) to show ranked results.

○ Become a Tutor

Definition	Record information tutors provide when applying to offer sessions.
Sub Items	• Account / Identity • Tutor Profile • Course You Can Tutor • Availability • Verification Documents • Policies
Input	Submitted in "Become a Tutor" form.
Usage	Enables tutor profile publication and matching with student requests.

3.) Functional Requirements - Prioritized

Priority 1:

- **FR-1.1 SFSU Registration:** Users must register with @sfsu.edu (suffix check only).

- Registered Users
 - Unregistered Users
 - Admin
- **FR-1.2 SFSU Sign-in:** Users must sign in with @sfsu.edu (suffix check only).
 - Registered Users
 - Unregistered Users
 - Admin
- **FR-5.1 Browse & Search:** Search for and browse different tutors by using the search function; options are data-driven and disabled/hidden when there are 0 matches.
 - Registered Users
 - Unregistered Users
- **FR-5.2 Filter:** Filter by Term, Session, Subject, Course Number, Location; options are data-driven and disabled/hidden when there are 0 matches.
 - Registered Users
 - Unregistered Users
- **FR-3 Create/Update Listing:** Tutor can add/edit headline, bio, subjects, course prefixes, availability, location tags, optional display-only rate.
 - Registered Users
- **FR-2 Tutor Role Enablement:** Students can enable a Tutor role to create listings.
 - Registered Users
- **FR-6 Listing Detail:** Detail page shows bio, subjects, course prefixes, availability, location tags, and Contact Tutor.
 - Registered Users
 - Unregistered Users
- **FR-7 One-Round In-Site Messaging:** Students can send one message to a tutor; no replies; no email clients.
 - Registered Users
- **FR-9 SFSU-Specific Discovery:** Course-centric filters, campus location tags, and semester-aware availability text.
 - Registered Users
 - Unregistered Users
- **FR-8 Role-Based Dashboards:** Student (Sent, profile); Tutor (Listings, Inbox, profile); Admin (Approval Queue, Moderation).
 - Registered Users
 - Admin
- **FR-4.1 Admin Approval:** Admin can approve/reject(with reason) certain listings and content.

Priority 2:

- **FR-4.2 Admin Moderation:** Admin can unpublish, delete listings; moderate users/content.
 - Admin

Priority 3:

- **FR-10 Analytics & Banner:** Google Analytics enabled; exact required banner text on every page.
 - Admin

4.) UI Storyboards

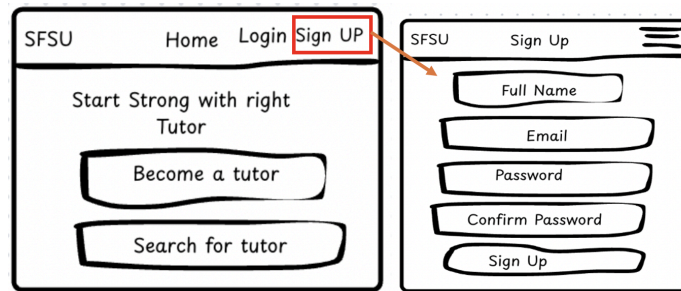
Use Case 1: User Registration

Description:

A new SFSU student or tutor creates an account to access the tutoring platform.

Storyboard (Mockups):

1. **Home Page** – The “Sign up” button is visible on the navbar.
User clicks “Sign up” to create a new account.
2. **Sign up Page** – form includes:
 - Full Name
 - SFSU Email
 - Password / Confirm Password
3. *User submits form → system validates input → sign up success.* **Success Page** – “Sign up successful! don’t need to login”



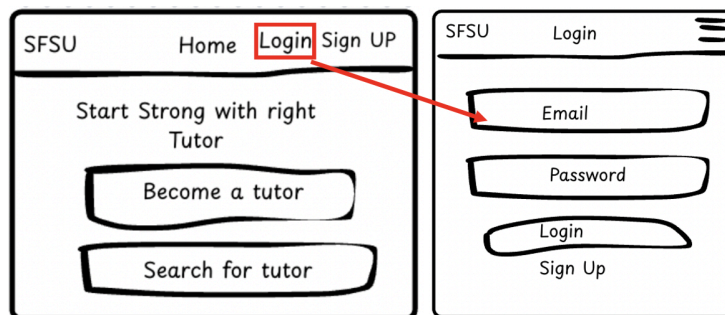
Use Case 2: User Login

Description:

Registered user logs in to access the tutoring dashboard.

Storyboard (Mockups):

1. **Login Page** – SFSU email + password input fields.
User clicks “Login.”
2. **Dashboard Page** – personalized homepage showing upcoming sessions or available tutors.



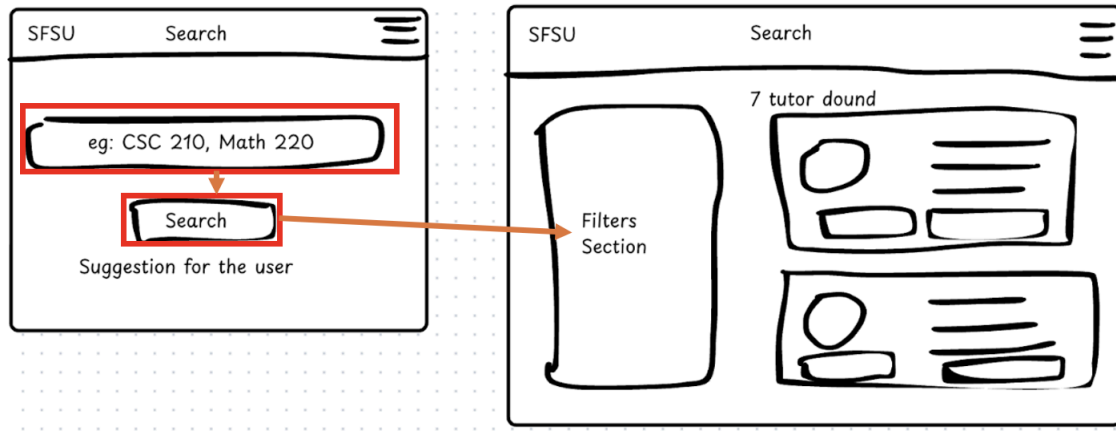
Use Case 3: Browse/Search for Tutors

Description:

Students can view available tutors and filter by subject, availability, or rating.

Storyboard (Mockups):

1. **Dashboard/Home Page** – “Search Tutors” button.
2. **Tutor Listing Page** – list of tutors with name, subject, availability, and “Book” button.
Optional: Apply filters according to the user's needs.



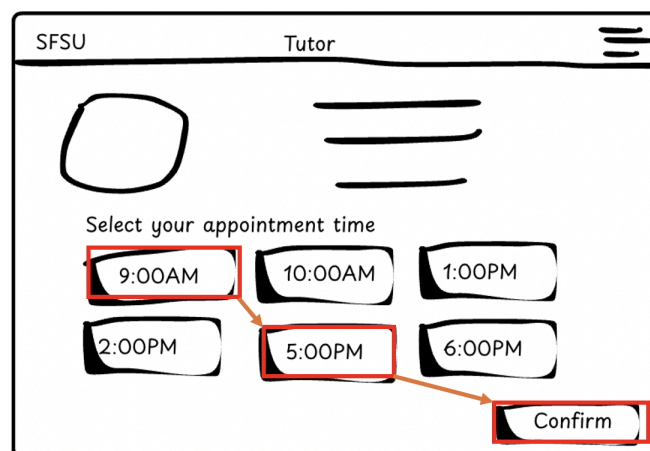
Use Case 4: Book a Tutoring Session

Description:

Students book a time slot with a tutor.

Storyboard (Mockups):

1. **Tutor Profile Page** – tutor’s info and schedule displayed.
User selects a time slot and clicks “Book.”
2. **Booking Confirmation Page** – shows selected date/time and confirmation button.
3. **Success Page** – “Your session has been booked successfully.”



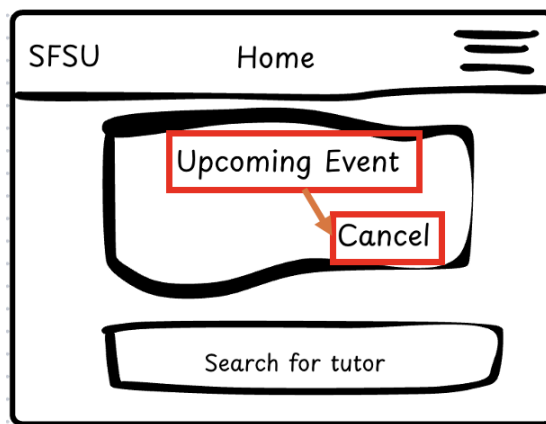
Use Case 5: View Upcoming Sessions (Dashboard)

Description:

Users view and manage their scheduled sessions.

Storyboard (Mockups):

1. **Dashboard Page** – table/list of sessions with columns:
 - Tutor Name
 - Date & Time
 - Subject
 - “Cancel” button (if allowed)
2. *The user can navigate to tutor details or cancel a session.*



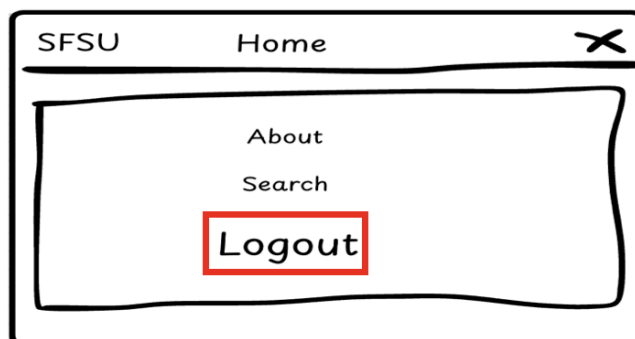
Use Case 6: Logout

Description:

User logs out of the system securely.

Storyboard (Mockups):

1. **Navbar/Menu** – “Logout” option available.
User clicks “Logout.”
2. **Redirect Page** – “You have been logged out” message with link to login page.



5.) High level Architecture, Database Organization summary only

The main tables that we will have in our database will be: User, TutorApplication, and Tutors (searchable).

For the user table, the items / columns we will have are: user id, name, email, password, user role, and registration date.

For the TutorApplication table, the columns we will store are application id, user id, headline, bio, meeting options, courses, availability, documents, application status, created at

For the Tutors table, we will have the following columns: id, user id, application id, slug, headline, bio, meeting options, courses. We will also have some columns about the tutor and their experience: hours tutored, rating, and how many ratings they have received.

Our media (pictures / videos) will be kept in the file system, specifically in the static/images folder.

For searching / filtering the database, we are using LIKE substring matching. It will return the closest match across slug/headline/bio/courses/meeting options. You can also look for tutors within a certain subject prefix or the exact course / location.

6.) Key Risks

Technical Risk:

Our main risk is ensuring the website is fully mobile responsive, since most of the current development focused on desktop-first UX design. Adapting layouts and components for smaller screens may require additional redesign and testing effort.

Resolution:

Adopt a hamburger menu below the md breakpoint to collapse non-critical links, while keeping 1–2 primary actions visible.

Schedule Risk:

Several members of ours are juggling other courses with overlapping deadlines, work or personal matters that could delay completion of milestones or managing work more efficiently.

Resolution:

Use a reliable task manager and check-ins on Fridays to surface any issues early as well as ensure fair production from every member

Skills Risk:

Some team members are still learning or are new to using Flask-MySQL integration and conventions, which may slow our backend process and deployment setup.

Resolution:

Backend lead and support will communicate frequently to align on process as well as reach out to other teammates for input or shared troubleshooting when needed. They'll also use online tutorials and some use of GenAI tools for self learning to strengthen our backend implementation.

Technical/Teamwork Risk:

While milestone 2 involves limited parallel work/development, later milestones will require multiple of our members to be contributing code simultaneously, which could lead to some merge conflicts or even working on the same things others might have already done or are doing at the same time.

Resolution:

Continue using clear branch naming and pull request reviews. A lot will fall on our github manager but active communication between the frontend and backend leads is expected to help prevent conflicts and redundant work. As well as to keep field names and folder structures consistent.

Legal/Content Risk:

Tutor photos or media used in mockups and website could unintentionally violate copyright or student privacy if sourced externally or without consent.

Resolution:

Use royalty free placeholder images from reliable sources, SFSU generated visuals, or our own generated visuals. Frontend team will be responsible for verifying that all visual assets used in the site comply with these norms before submission.

UX / Usability Risk:

If our interface or flow is confusing, users might get frustrated or not want to continue using the site especially when it comes to trying to find a tutor or complete basic options.

Resolution:

Have team members test the mockups and early builds to spot any confusing layouts or even any areas to improve. That way small design adjustments can be given to the front end team to fix based on that feedback. Ensure it's a smooth experience for users by acting like one yourself.

7.) Project Management

ClickUp will function as a unified dashboard for each team member to view and track the status of all assigned tasks. Every task has an owner, due date, priority, a checklist lifted from the M2 PDF, and grouped by the M2 sections (Data Items, Functional Reqs, UI Storyboards, Architecture/DB) with applicable dependencies so the critical path is clear. The status flow of each task will follow 'To Do → In Progress → In Review → Done'. Team lead will review each task prior to moving the status to 'Done'. Primary communication will be conducted via discord, with regular check-ins prior to the weekly team meeting on Friday.

The front end owns: the page templates (Jinja/HTML), the visual design (CSS), and small, focused JavaScript for UI details (the hamburger menu, chips, counters). The back end owns: Flask routes and APIs, database models and queries (SQLAlchemy), authentication and sessions, validation, file uploads, search/filter/pagination, and security. Common interfaces will be agreed upon (each page or API we agree on the URL, inputs, and the exact field names returned to templates/JSON (matching the data names defined in Section 2).

8.) Use of GenAI

For Milestone 2, our team's use of Generative AI (GenAI) tools was limited to light research and clarification support, rather than direct content generation. Most written content, including requirements, UI storyboards, and architecture descriptions, was drafted collaboratively through group discussion and manual editing. However, we recognize the potential of GenAI tools (such as ChatGPT, GitHub Copilot, or others) in supporting different aspects of the software engineering lifecycle, and we plan to integrate them more actively in upcoming milestones.

Specifically, we anticipate using GenAI in the following ways:

- Code Assistance & Debugging: In order to write and fix small portions of our code more quickly, we intend to use GenAI tools.
- Documentation & Comments: For our project, GenAI can assist us in creating concise documentation and clear comments.
- UI/UX Feedback and Copy Suggestions: To enhance the user experience, we might employ GenAI to recommend better wording or layouts.
- Brainstorming & Risk Management: GenAI can help us generate innovative ideas and identify potential project risks early.

9.) Team Lead Checklist

Checklist Item	Status	Comments
So far, all team members are fully engaged and attending team sessions when required	OK	Consistently engaged via discord & attending classes.
Team reviewed suggested resources before drafting Milestone 2	OK	
Team lead checked Milestone 2 document for quality, completeness, formatting and compliance with instructions before the submission	OK	

Team lead ensured that all team members read the final Milestone 2 document and agree/understand it before submission	OK	
Team shared and discussed experience with GenAI tools among themselves	OK	

Milestone 3

1. UI / UX Feedback

- The search button on the home page currently requires extra clicks, which may reduce usability. The search flow should be streamlined to minimize user actions.
- Some dropdown menus have inconsistent sizing, which affects visual alignment and clarity.
- When multiple filters (e.g., Math → 256 → Zoom) are selected, the page refreshes too frequently or resets unexpectedly. The system should avoid premature or double refreshes and only update results when necessary.
- Dynamic truncation was recommended. Once a user selects a high-level filter (e.g., subject), subsequent filter options should narrow accordingly.
- Pagination on the tutor dashboard (e.g., “1 2 3”) should be evenly aligned across the page for better visual balance.
- The label “Courses I Tutor” currently resembles a visual divider rather than an information label. A clearer and more explicit label should be used.
- Hover animations should only apply to clickable elements to avoid confusing users.

2. Code and Architecture Feedback

- The current tutor availability grid appears cluttered and visually complex. A simpler row-based layout (day, start time, end time, modality) was recommended.
- Tutor availability should be stored as an array of entries rather than multiple individual input fields, improving maintainability and extensibility.
- Availability input should only be shown or required after a tutor has been approved, as availability may change frequently.
- The design should support future flexibility, such as different modalities on different days (e.g., Zoom on Mondays, in-person on Tuesdays).
- The professor emphasized prioritizing workflow clarity and maintainable architecture before introducing additional complexity.

3. GitHub Usage Feedback

- The team should continue committing frequently with small, well-defined changes.
- Branch naming conventions and pull request practices should remain consistent.
- All team members should be able to pull the latest version and run the project locally or on the server.
- Documentation in the repository should be updated after each meeting.

4. Database Feedback

- No major structural issues were identified at this stage.
- As backend functionality expands, remaining features should be carefully planned to prevent schema drift.
- Search filters should be tested to avoid unnecessary database queries that may cause excessive refreshes.

- Relationships between **User**, **TutorApplication**, and **Tutor** tables should be validated to ensure consistency after approval or rejection.

5. Teamwork and Risk Management Feedback

- Two dashboards remain in scope: Tutor Dashboard and Admin Dashboard.
- Backend implementation should begin while front-end members continue refining UI.
- At least one complete round of messaging functionality should be implemented before Milestone 4.
- Clear task division and strong planning are essential as development complexity increases.
- Risk reminder: if the EC2 server crashes, use **htop** to check CPU and RAM usage before restarting, as resource limits are a common cause. Contact the instructor if needed.

6. Agreed-Upon P1 Features for Final Delivery

1. Search Functionality

- Reduce the number of clicks on the home page search.
- Fix refresh behavior when multiple filters are applied.
- Implement dynamic truncation of filter options.

2. Tutor Dashboard

- Display tutors with clean and consistent pagination alignment.
- Standardize layout, spacing, and fonts; update the “Courses I Tutor” label.
- Show availability only for approved tutors.

- Redesign availability using a row-based layout.

3. Admin Dashboard

- Approve or reject tutor applications with clear status indicators.
- Display summary counts (pending, approved, rejected).
- Provide streamlined controls for promoting users to tutor or admin roles.

4. Authentication and Roles

- Maintain functional login and signup with [@sfsu.edu](mailto:sfsu.edu) validation.
- Ensure role-based access control across all views.

5. Messaging Functionality

- Implement at least one full message exchange between tutor and student.

6. Database Integrity

- Maintain stable relationships between core tables.
- Ensure approved applications correctly update tutor listings.

7. UI / UX Enhancements

- Match dropdown sizes and restrict hover effects to interactive elements.
- Improve responsiveness on both mobile and desktop devices.

8. Server Stability and Deployment

- Ensure reliable deployment on AWS EC2 using Flask, Gunicorn, and Nginx.
- Monitor server resources to prevent crashes.

7. Additional Notes

- Architecture review confirmed that the MVC structure, naming conventions, and code comments are appropriate.

- No additional issues were identified at this time.

Milestone 4

1.) Product Summary

Product Name: GatorMatch Tutors (<http://18.191.185.168/>)

Product Description (Plain English, 1 paragraph)

GatorMatch Tutors is a safe, simple, SFSU-only platform that helps students quickly find credible peer tutors for the exact courses they're taking. Any @sfsu.edu student can search by course code (e.g., CSC 340), browse approved tutor listings, check availability, and send a one-round in-site message to connect—no outside email or messaging apps required. Tutor profiles highlight meaningful trust signals such as course expertise, availability, campus meeting options, and SFSU-verified eligibility. Tutors can easily create and update listings, and every listing goes through an Admin approval step to ensure accuracy and safety. What makes GatorMatch unique is its SFSU-specific design: course-centric search, campus location tags, semester-aware availability, and filters that never show empty results. The platform provides a moderated, campus-focused directory that is easy for students, tutors, administrators, and faculty to rely on.

P1 Student Functions

6. Students can register using an @sfsu.edu email address.
7. Students can sign in with an @sfsu.edu email address.
8. Students can browse and search for tutors using data-driven filters that hide options with zero matches.
9. Students can view full tutor listings including bio, subjects, courses, availability, and location tags.
10. Students can send a one-round in-site "Contact Tutor" message (no replies allowed).

P1 Tutor Functions

9. Students can enable a Tutor role to create tutor listings.

10. Tutors can create or edit their listings, including headline, bio, subjects, course prefixes, availability, and location tags.
11. Tutors can submit listings for Admin approval and see their status (Draft, Pending, Approved, Rejected).

P1 Admin Functions

11. Admins can approve or reject listings and provide rejection reasons.
12. Admins can access a role-based dashboard with an approval queue and moderation tools.

P1 System-Level Features

13. Course-centric and SFSU-specific discovery (course filters, campus locations, semester-friendly availability).
14. Role-based dashboards for Students, Tutors, and Admins.

2.) Usability Test Plan for Selected Function

2.1 Test Objectives

The objective of this usability test is to determine whether users can easily and efficiently find a suitable tutor using the Search Tutors function. We want to see how well students can locate search tools, apply filters, interpret results, and reach a tutor's detail page without confusion. This test helps us identify unclear labels, possible navigation issues, steps that cause hesitation, and how often users successfully complete the task without help. The results will show whether our search interface is intuitive for new SFSU students and whether the experience supports their goal of quickly finding academic assistance.

2.2 Test Background and Setup

This usability test will be conducted on the deployed version of GatorMatch Tutors. The system will run on a laptop or phone with a stable internet connection and at least two major browsers installed, such as Google Chrome and Safari. The test database will contain a set of realistic tutor listings across different courses so that search results behave naturally. Participants begin on the home page of the website, and no registration is required for the test. Our intended users are SFSU students who may need tutoring, since they best represent the real audience for this function. The test will take place in a quiet environment, and

the facilitator will not assist unless the participant is stuck for more than three minutes. The deployment URL used for testing will be the team's public server link.

2.3 Usability Task Description (Instructions to Tester)

In this scenario, the participant acts as an SFSU student who is currently struggling with CSC 340 and wants to find a tutor who is available either on campus or on Zoom. Starting from the home page, the participant must navigate to the page that shows all tutors, use the search bar or filters to find tutors who can help with CSC 340, and then apply at least one additional filter such as subject or location. After finding a suitable match, the participant should choose one tutor, open their detail page, and describe why they selected that tutor. Finally, the participant must identify where they would click to contact the tutor, although they do not need to send an actual message.

2.4 Plan for Evaluation of Effectiveness

To evaluate effectiveness, we will observe whether the participant can complete the full task without assistance: locating the tutor listings page, performing a course-specific search, applying additional filters correctly, opening a relevant tutor's detail page, and locating the Contact button. We will record whether each step is completed with no help, completed with minor hints, or not completed at all. We will also document errors such as selecting the wrong page or applying filters incorrectly. The overall effectiveness will be measured by the percentage of participants who successfully finish the entire task within a ten-minute time limit.

2.5 Plan for Evaluation of Efficiency

Efficiency focuses on how quickly and smoothly participants complete the scenario. We will measure the total time from the moment they start on the home page to when they identify the Contact button on the selected tutor's detail page. We will also count how many clicks and page transitions the participant makes and note whether they need to undo steps, reset filters, or navigate back. After testing multiple users, we will calculate the average completion time, fastest and slowest times, number of clicks, and identify any patterns that suggest confusing design elements.

2.6 Plan for Evaluation of User Satisfaction

After completing the task, each participant will answer a short questionnaire using a 5-point Likert scale to measure satisfaction. The questions ask whether it was easy to find a tutor, whether the search and filter options were clear, whether the information provided was useful, whether the participant would feel comfortable using this website in real life, and whether they were satisfied with the overall experience. Participants may also answer two optional open-ended questions about what they liked most and what they believe should be improved. These responses help us understand the emotional and subjective side of the user experience.

2.7 GenAI Use (Usability Test Plan)

For this section, we used a Generative AI tool to help refine the clarity and structure of our usability test plan. ChatGPT reviewed our early drafts and suggested clearer task phrasing, better separation between effectiveness and efficiency metrics, and improvements to the wording of our Likert-scale questions to ensure neutrality. We asked prompts such as “Please improve the clarity of this usability test scenario” and “Are these survey questions neutral and appropriate?” The tool was helpful for editing and organizing our ideas, so we consider the utility of GenAI in this section to be medium.

3.) QA Test Plan and QA Testing

3.1 Test Objectives

- Verify that the **Search Tutors** feature works correctly for typical and edge-case inputs.
- Confirm that search and filters return matching tutors and handle “no result” cases properly.
- Check that behavior is consistent across at least two major web browsers.

3.2 Hardware and Software Setup

- Hardware:
 - Laptop/phone with stable network connection.
- Software:
 - Operating system: Windows/macOS/Linux (any current version).
 - Browsers:
 - Browser A: Google Chrome (latest version).
 - Browser B: Safari (latest version).
 - Backend: Flask app connected to MySQL database as in project architecture.
 - Deployed server URL: <http://18.191.185.168/>

3.3 Feature to Be Tested

- **Browse & Search Tutors** including:

- Free-text search by course or keyword.
- Filters by subject, course number, location, and term/session.
- Display of tutor listings and navigation to detail pages.

3.4 QA Test Plan Table (Executed on Two Browsers)

Test	Test Title	Test Description	Test Input	Expected Correct Output	Browser A (Chrome)	Browser B (Safari)
T1	Basic course search	Ensure searching by a valid course shows correct tutors	CSC 340	At least one tutor with CSC 340 appears	PASS	PASS
T2	Search no results	Ensure system handles empty results properly	ABC 878	“No tutors found” message appears	PASS	PASS
T3	Combined filters	Test subject + location filtering	Subject=CS C, Location=Zoom	All tutors shown match CSC + Zoom	PASS	PASS
T4	Open detail page	Check navigation to tutor details	Click first tutor	Correct tutor detail page opens	PASS	PASS

3.5 Execution Notes (Two Browsers)

We ran all four QA tests on Google Chrome and Safari. Every test worked the same on both browsers, and all results were correct. The search feature showed the right tutors, the “no results” message appeared

when expected, filters worked properly, tutor detail pages opened with no issues, and long inputs did not break the page. There were no errors or layout problems in any test. All tests passed on both browsers.

3.6 GenAI Use (QA Test Plan)

Tool used

- ChatGPT

How we used the tool and benefits

- We gave the ChatGPT a short description of our search feature and asked it to suggest concrete test cases including normal, edge, and error inputs.
- It helped us think of specific cases like very long search strings and no-result scenarios, and reminded us to test across multiple browsers and handle empty states clearly.

Example prompts

- “We have a Flask web app where students search for tutors by course and filters. Suggest a QA test plan with at least 5 test cases in table format, including normal input, invalid input, and no results.”
- “Please adjust these test cases so they are easy to read by non-technical stakeholders and include expected outputs.”

4.) Peer Code Review (with GenAI)

4.1 Selected Code for Review

For this Milestone, our team selected the backend code supporting the **Browse & Search Tutors** feature and the **Tutor Session Request** flow. The following files were included as part of the review:

- app.py
 - Routes for tutor search, tutor profiles, session requests, dashboards
- models.py
 - SQLAlchemy models for User, Tutor, TutorApplication, Tutor, TutoringSession, Availability Blocks
- forms.py
 - Registration, login, tutor application forms, custom validators

- `admin.py`
 - Admin dashboard, approval/rejection flows
- `db.py`
 - SQLAlchemy engine and session configuration
- `jitsi_helper.py`
 - Helper for generating Jitsi meeting links

This code represents the backend logic responsible for finding tutors, validating SFSU-only usage, storing tutor data, handling applications, and creating tutoring sessions.

4.2 Peer Review Process (Human Reviewer)

1. Submission of code for review

- The backend developer (e.g., Derek) sent a link to the relevant GitHub branch/file to another team member (e.g., Stanley) via email/Discord, requesting a formal code review.

2. Review activities performed

The reviewer checked for:

- Clear and consistent naming of variables, functions, and models (aligned with data definitions in Milestone 2).
- Presence of header comments and helpful inline comments where logic is complex.
- Proper handling of user input (sanitizing or parameterizing queries).
- Separation of concerns between route logic and template rendering.
- Readability and maintainability (avoiding deeply nested conditionals, repeated code, etc.).

3. Feedback and communication

- The reviewer left comments directly in GitHub (or equivalent) pointing out:
 - Suggested renaming of some variables for clarity.
 - Opportunities to extract repeated filter conditions into helper functions.
 - Small style issues such as spacing and docstring formatting.
- A summary of review findings was sent back via email or team chat, including any “must-fix” issues before merging.

4.3 Peer Review Using GenAI (Required)

How GenAI Was Used

We provided ChatGPT with the relevant backend files (app.py, models.py, forms.py, admin.py, jitshi_helper.py) and asked it to:

- Review program structure and clarity
- Identify potential security issues
- Suggest improvements in maintainability and refactoring
- Analyze database relationships
- Evaluate validation logic and error handling
- Highlight missing or incomplete feature logic

This helped verify architectural choices and catch inconsistencies that human reviewers may overlook.

GenAI Review Findings (Summary)

Strengths Identified by GenAI

- Good modular structure and clean ORM model definitions
- Secure password hashing and input validation
- Well-designed tutor and session models
- Safe DB usage through scoped sessions

GenAI's Suggested Improvements

- 1. Implement missing search logic**
GenAI noted that /search currently does not process query parameters, which does not align with the QA test plan.
- 2. Enforce admin-only access in all admin routes**
GenAI recommended a decorator-based solution.
- 3. Replace magic strings with Enums or shared constants**
Improves consistency and future extensibility.

4. **Improve date/time parsing reliability in session creation**
GenAI recommended explicit error messages and standardized parsing.
5. **Add docstrings to several routes**
Improves comprehension for future developers.
6. **Consider extracting session logic into a service layer**
Reduces the size of route handlers and improves testability.

Utility Ranking of GenAI in This Code Review

RANK: HIGH

GenAI provided:

- Precise identification of architectural issues
- Actionable recommendations aligned with best practices
- Useful refactoring suggestions
- Security validations that reinforced our own review
- Improvements that directly help with Milestone 4 and final project quality

GenAI significantly accelerated the review process and ensured a comprehensive evaluation beyond what a single teammate could perform manually.

5.) Security Self-Check

5.1 Assets, Threats, and Mitigations

Asset	Possible Threats	Impact if Breached	Mitigation (Protection Method)
User accounts & passwords	Stolen passwords, brute-force login	Unauthorized access to user data	Store hashed passwords; limit login attempts

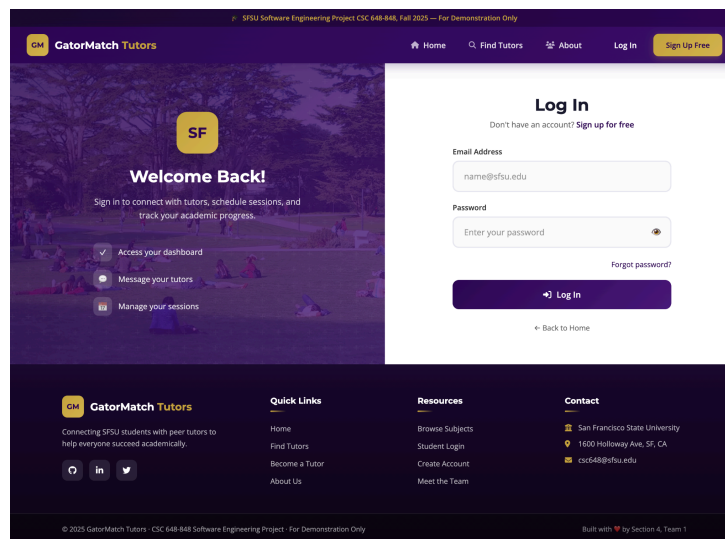
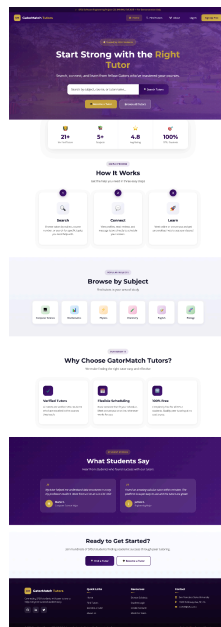
User profile info (name, SFSU email)	Data leaks, scraping	Privacy issues, spam	Restrict access; do not show emails publicly
Tutor listings	Unauthorized edits or deletion	Wrong or missing information	Allow edits only by tutors/admins; require admin approval
One-round messages	Spam or misuse	Harassment or abuse	Log messages; keep messages inside the site
Server & database	SQL injection, unauthorized access	Full system compromise	Use parameterized queries; secure SSH; protect environment variables

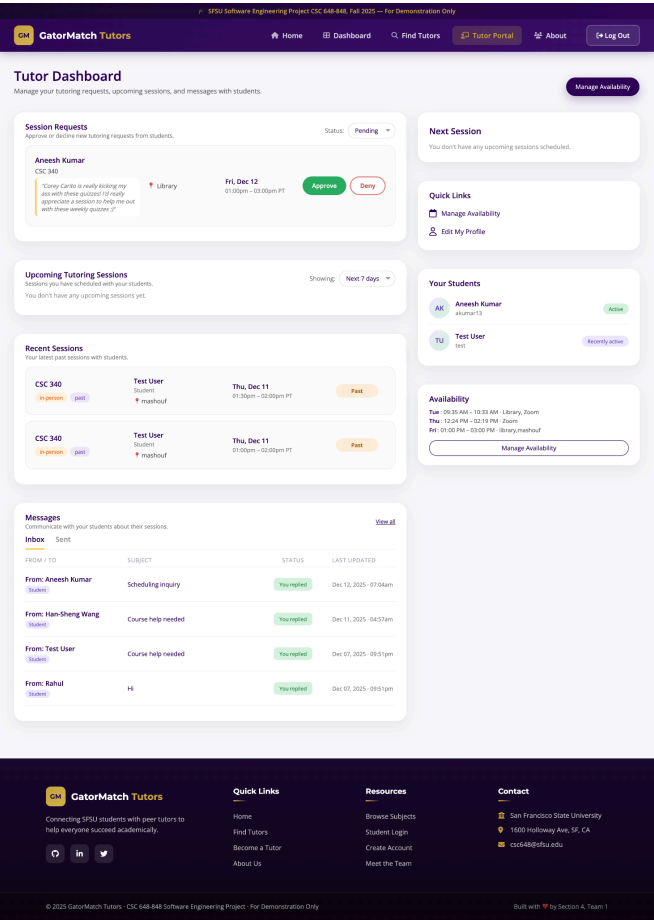
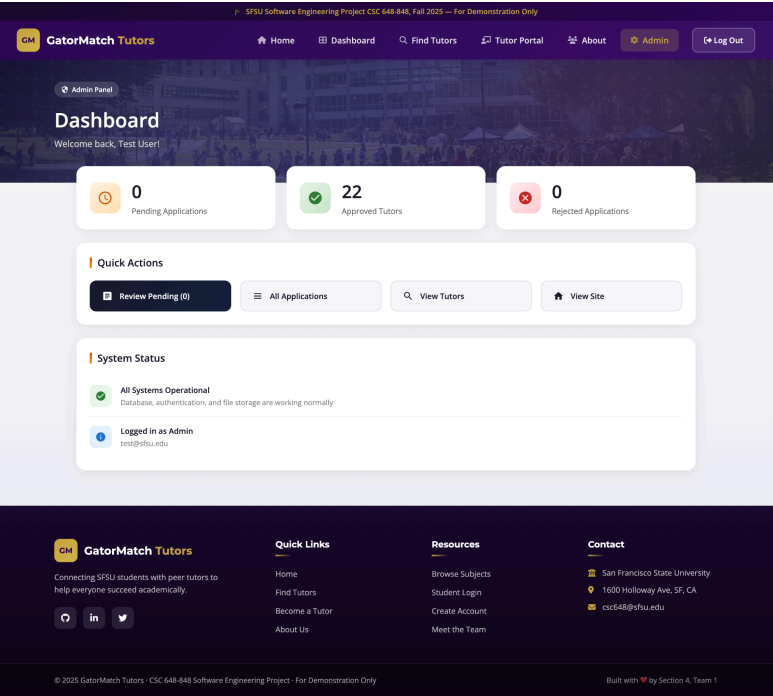
6.) Self-Check of the Adherence to Original Non-Functional Specs

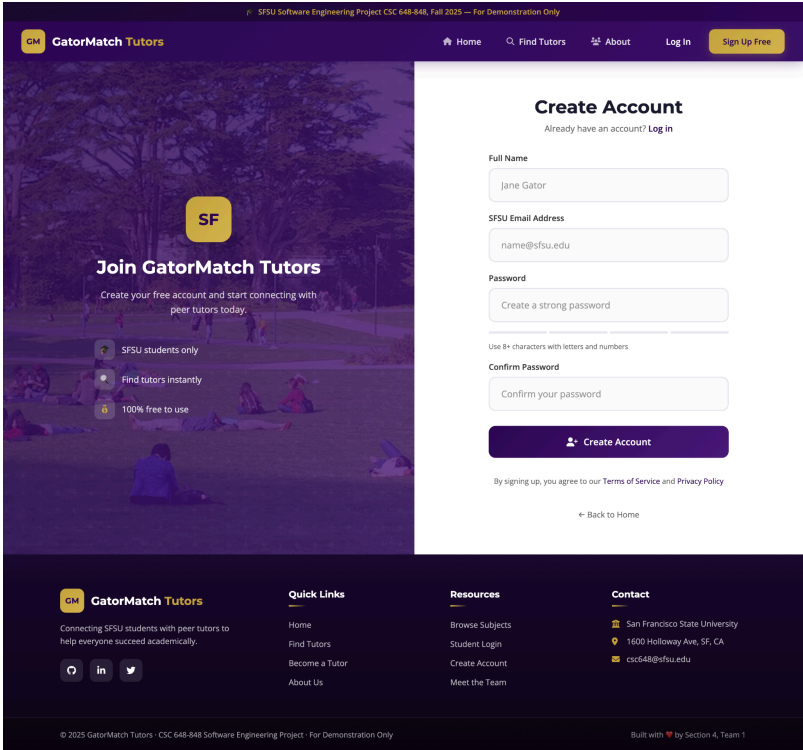
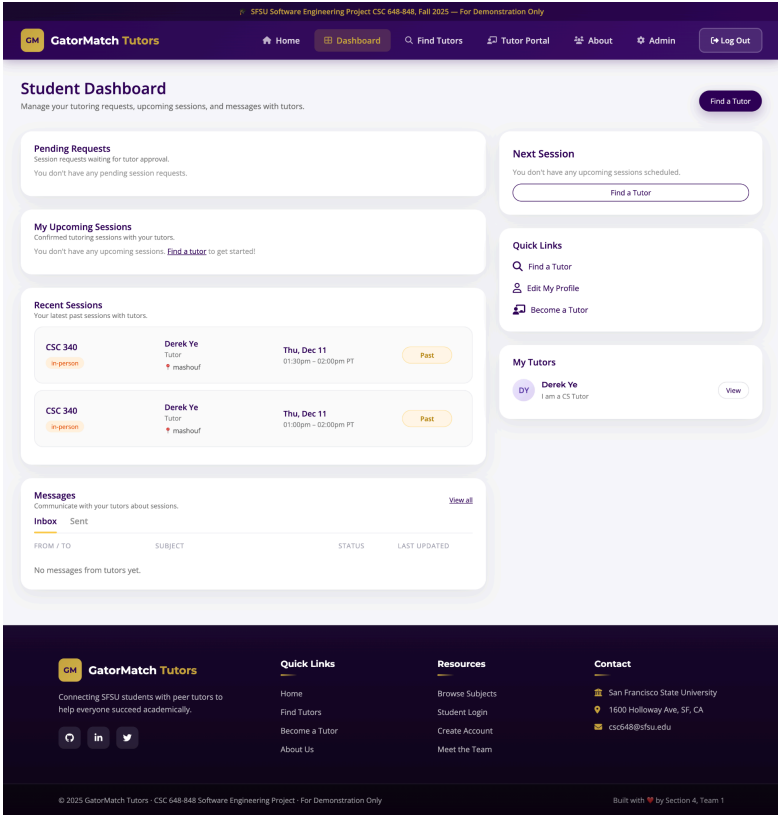
#	NFR	Status	Notes
1	Use approved tools/servers (per Class CTO, M0)	DONE	
2	Optimized for standard desktop/laptop browsers (latest 2 versions of 2 major browsers)	DONE	
3	Key functions render well on mobile devices (no native app)	DONE	
4	Tutor posting & messaging limited to SFSU students	DONE	
5	Critical data stored in DB on team's deployment server	DONE	
6	No more than ~50 concurrent users expected	DONE	
7	User privacy is protected	DONE	
8	English-only UI (no localization)	DONE	
9	Application very easy and intuitive to use	DONE	
10	Follows established architecture patterns	DONE	
11	Code & repository easy to inspect and maintain	DONE	
12	Google Analytics used	DONE	
13	No email clients; only in-site messaging; single message round; no chat	DONE	

14	No payment functionality implemented or simulated	DONE	
15	Basic site security best practices applied to main data items	DONE	
16	Media formats are standard (as used in the market today)	DONE	
17	Modern SE processes & tools (collab, CI/CD, GenAI, etc.) used as specified in class	DONE	
18	UI displays required disclaimer text on all pages in navbar	DONE	

3) Product Screenshots







GatorMatch Tutors

[Home](#)
[Dashboard](#)
[Find Tutors](#)
[Tutor Portal](#)
[About](#)
[Log Out](#)

Become a Tutor

Share your knowledge and help fellow SFU students succeed. Join our community of tutors today.

[Start Application](#)
[View Requirements](#)

Account & Identity

Your SFU account information

Full Name

Check this

SFU Email

SFU email is required to become a tutor

Requirements

- Valid SFU email address
- Minimum 90% grade in courses
- Proof of grades (transcript/report)
- Available for campus or Zoom

Pro Tip

Write a clear headline and can-do list. Students filter by campus and Zoom... select both to appear in more searches!

[Review Time](#)

Applications are typically reviewed within 48-72 hours.

Tutor Profile

Tell students about yourself!

Headline

E.g. "CSC 340 Tutor - Data Structures & Algorithms"

400 characters

Bio

Tell students about your experience and teaching style (3-5 sentences)

80 characters

Meeting Options

☒ SFU Library ☐ Mashout Center ☒ Zoom (Home)

Courses You Can Tutor

Add courses you've completed with a 90% grade

Select a course...

[Add](#)

Verification Documents

Upload transcript or grade reports (PDF, PNG, JPG -- max 10MB each)

Document 1 *

Click to upload or drag & drop

Document 2 (optional)

Click to upload or drag & drop

Document 3 (optional)

Click to upload or drag & drop

Policies & Agreement

☐ I certify that the information provided is accurate and I agree to comply with [SFU academic honesty policies](#) and tutoring guidelines.

[Submit Application](#) [I Save Draft](#) [Clear](#)

GatorMatch Tutors

[Home](#)
[Dashboard](#)
[Find Tutors](#)
[Tutor Portal](#)
[About](#)

Connecting SFU students with peer tutors to help everyone succeed academically.

[G](#) [i](#) [t](#)

Quick Links

- Home
- Find Tutors
- Become a Tutor
- About Us

Resources

- Browse Subjects
- Student Login
- Create Account
- Meet the Team

Contact

San Francisco State University

1601 Holloway Ave, 3rd Fl., CA

csc448@sfu.edu

© 2025 GatorMatch Tutors - CSC 448-848 Software Engineering Project - For Demonstration Only

Built with by Section 4, Team 1

GatorMatch Tutors

[Home](#)
[Dashboard](#)
[Find Tutors](#)
[Tutor Portal](#)
[About](#)
[Log Out](#)

Manage Availability

Set your weekly hours so students know when they can book sessions with you.

Weekly Schedule

Toggle days on/off and add your available time blocks

Monday Unavailable

Toggle "Available" to add time slots for this day

Available ☐

Tuesday Live

Toggle "Available" to add time slots for this day

Available ☒

FROM: 06:00 TO: 06:00 LOCATION: Library Mashout Zoom

COURSES FOR THIS TIME SLOT: CSC 340

[+ Add Time Slot](#)

Wednesday Unavailable

Toggle "Available" to add time slots for this day

Available ☐

Thursday Live

Toggle "Available" to add time slots for this day

Available ☒

FROM: 06:00 TO: 06:00 LOCATION: Library Mashout Zoom

COURSES FOR THIS TIME SLOT: CSC 340

[+ Add Time Slot](#)

Friday Live

Toggle "Available" to add time slots for this day

Available ☒

FROM: 13:00 TO: 15:00 LOCATION: Library Mashout Zoom

COURSES FOR THIS TIME SLOT: CSC 340

[+ Add Time Slot](#)

Saturday Unavailable

Toggle "Available" to add time slots for this day

Available ☐

Sunday Unavailable

Toggle "Available" to add time slots for this day

Available ☐

Student Preview

This is how students see your availability

☒ Tue 09:30 - 10:30
 ☒ Thu 12:24 - 14:19
 ☒ Fri 13:00 - 15:00

GatorMatch Tutors

[Home](#)
[Dashboard](#)
[Find Tutors](#)
[Tutor Portal](#)
[About](#)

Connecting SFU students with peer tutors to help everyone succeed academically.

[G](#) [i](#) [t](#)

Quick Links

- Home
- Find Tutors
- Become a Tutor
- About Us

Resources

- Browse Subjects
- Student Login
- Create Account
- Meet the Team

Contact

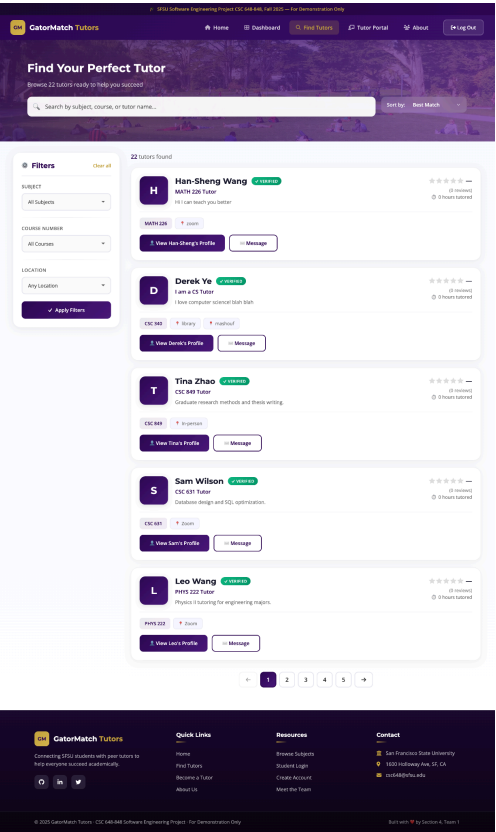
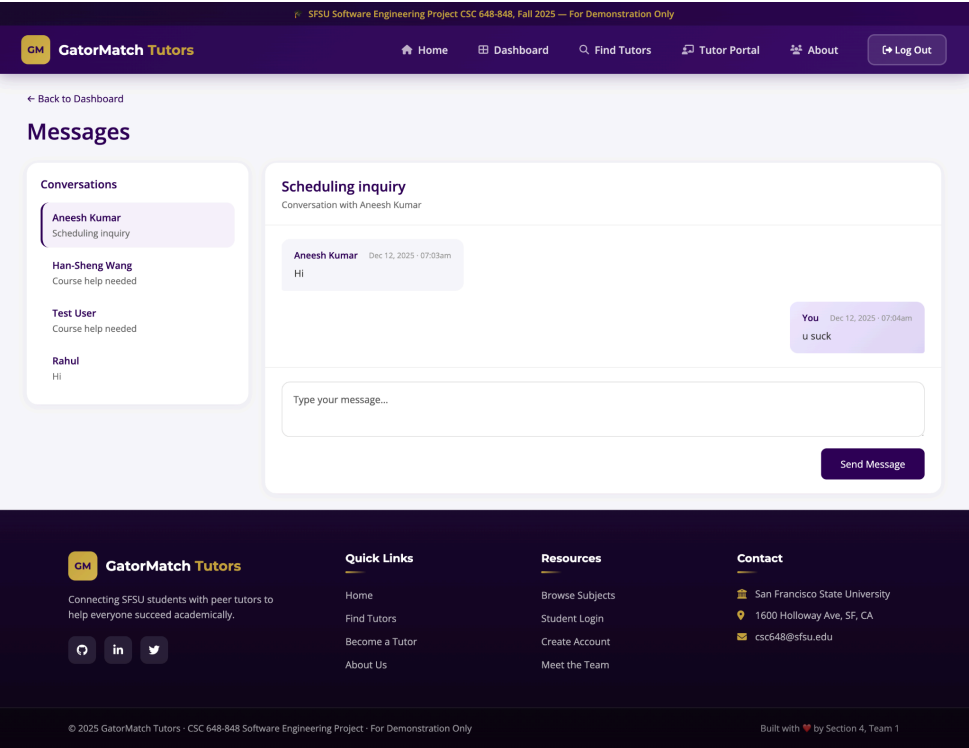
San Francisco State University

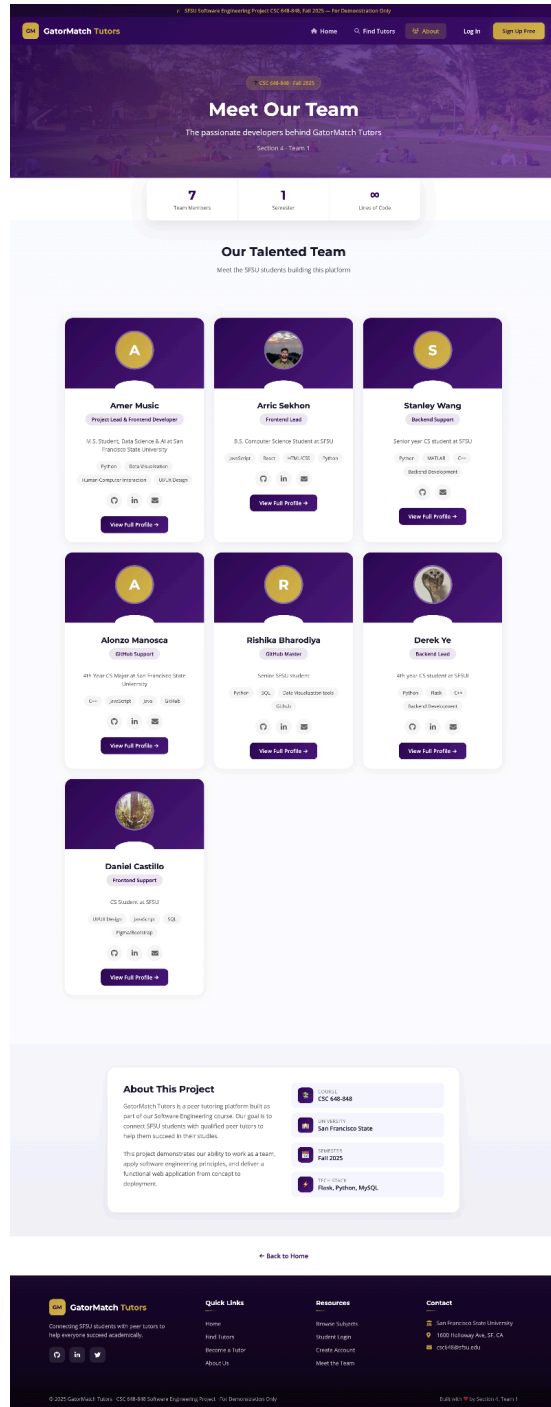
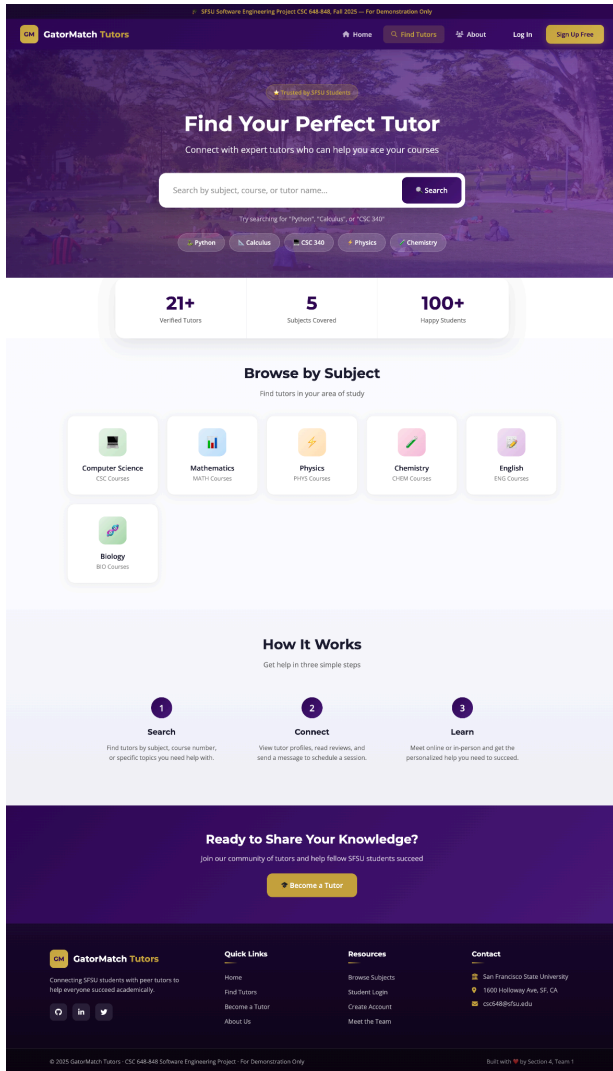
1601 Holloway Ave, 3rd Fl., CA

csc448@sfu.edu

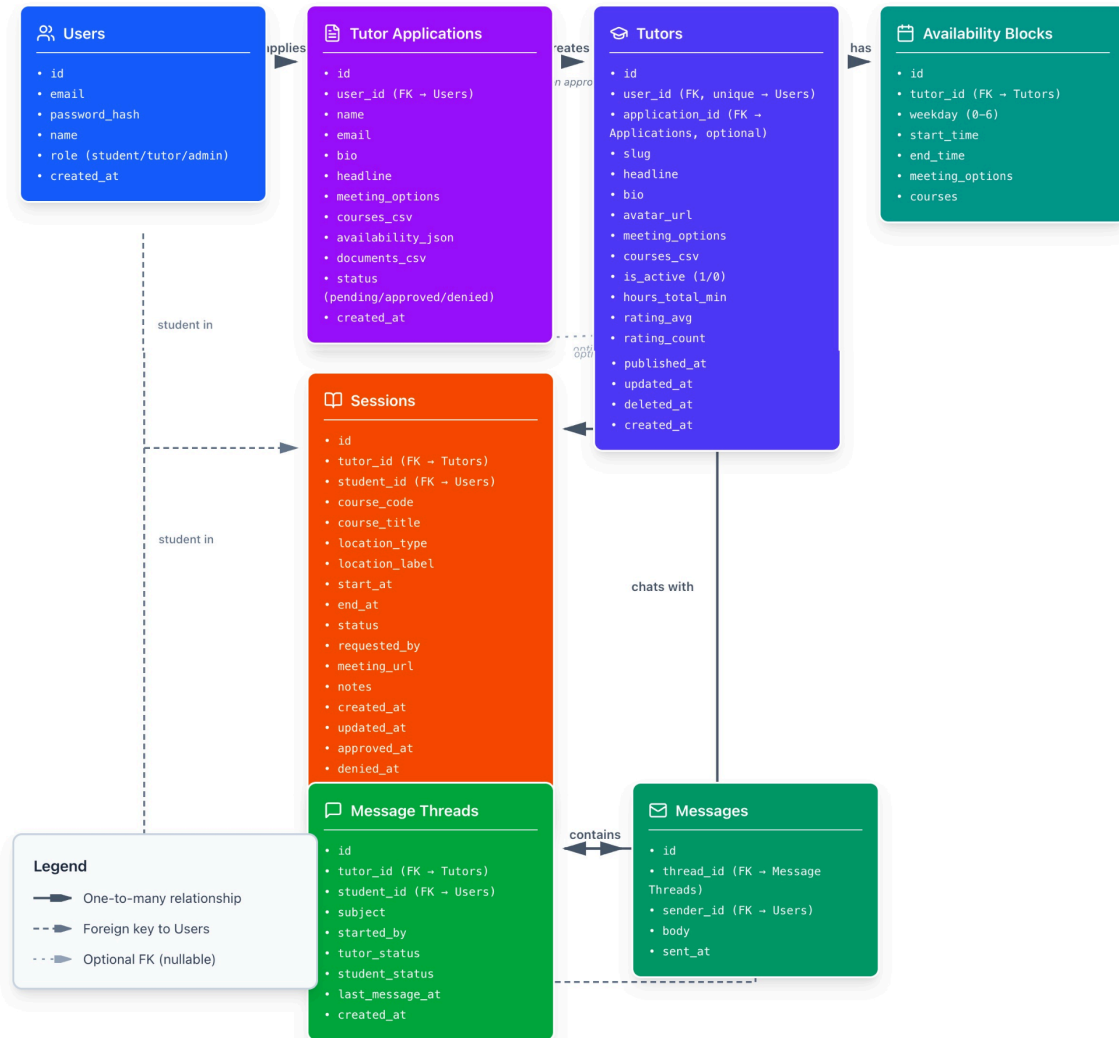
© 2025 GatorMatch Tutors - CSC 448-848 Software Engineering Project - For Demonstration Only

Built with by Section 4, Team 1





4) Database Organization



5) GitHub Organization

a) Main Branches in Team GitHub Repository

Our team GitHub repository contains the following primary branches:

- **main**
The main branch serves as the stable integration branch and contains tested, working code aligned with milestone requirements.
- **feature branches (multiple)**
Feature branches are used by individual team members to implement specific

functionalities (e.g., dashboards, messaging, UI updates, backend logic). These branches are merged into `main` through pull requests after review.

At the time of this milestone, the repository contains 17 branches, reflecting parallel development and feature-based workflows.

b) Access to Master/Main Branch

Access to the `main` branch is restricted to team members with repository write permissions. All contributors work on feature branches and merge changes into `main` through GitHub's standard workflow. This ensures:

- Controlled integration of new features
- Reduced risk of breaking the stable codebase
- Clear tracking of changes through commits and pull requests

No direct edits are made to the `main` branch without proper authorization.

c) Screenshot of Team GitHub Home Page

The screenshot shows the GitHub interface for a repository named 'csc648-fa25-145-team01'. The repository is private and was forked from 'CSC-648-SFSU/csc-648-sfsu-csc-648-848-section-01-04-05-fall-2025-petkovic-csc648-base-repo'. The repository has 0 watches, 0 forks, and 0 stars.

The main branch is selected, and it is 135 commits ahead of the upstream 'main' branch. The repository contains 17 branches and 0 tags. The 'Code' button is highlighted in green.

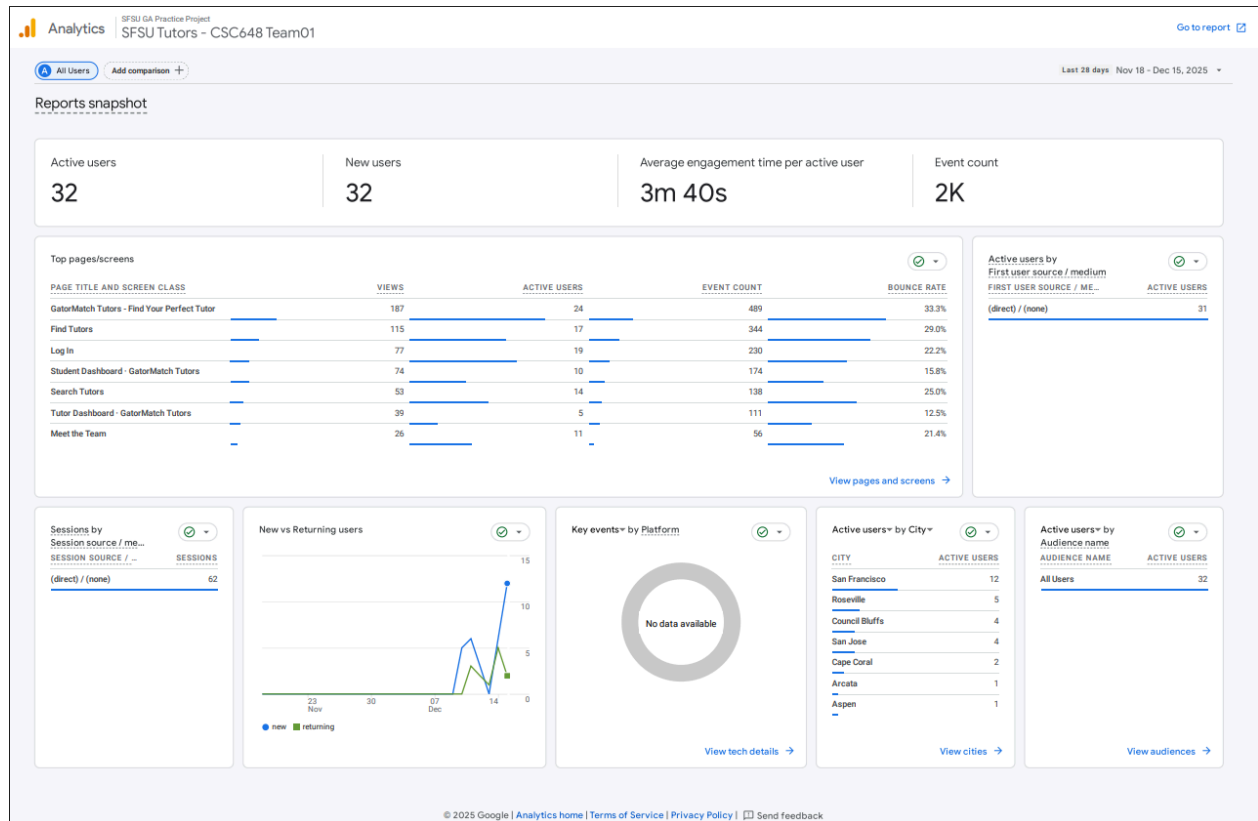
The commit history shows a commit by 'dacastillo516' titled 'added the time change to dashboards' from yesterday, with 136 commits in total. The commit message is '7745b17 · yesterday 136 Commits'.

The file list includes:

- Milestones: Add Milestone 4 Documentation (yesterday)
- application: added the time change to dashboards (yesterday)
- credentials: Update README with new server IP address (last month)
- .env.example: Make server host and port configurable via environment vari... (3 months ago)
- .gitignore: Update .gitignore and db.py (last month)
- LICENSE: Initial commit (4 months ago)
- README.md: Update README.md (3 months ago)
- requirements.txt: fixed timezone issue with messages and added tzdata to req... (yesterday)

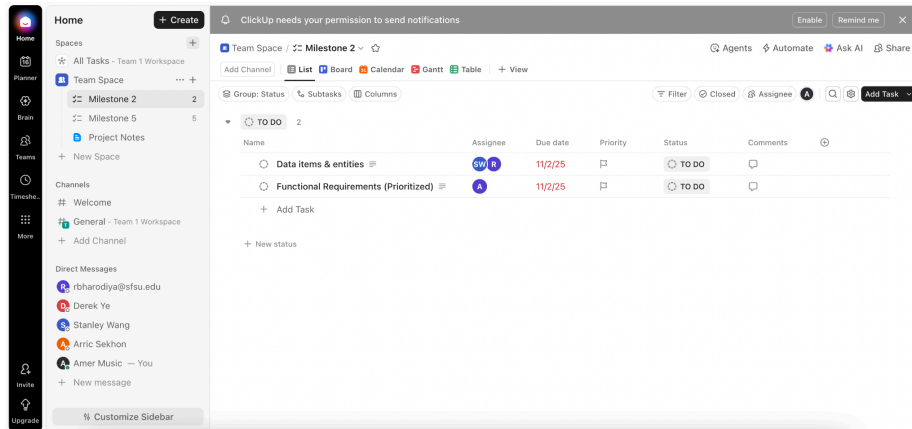
The right sidebar shows the repository's 'About' section, including the README, MIT license, activity, custom properties, 0 stars, 0 watching, and 0 forks. The 'Releases' section shows no releases published, with a link to 'Create a new release'. The 'Packages' section is also visible.

6) Google Analytics Stats Plot for Your WWW Site



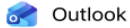
7) Project Management

In addition to consistent communications among group members on discord, we formally assigned tasks and deadlines using Clickup (see screenshot below).



8) Team Member Self-Assessment and Contributions:

1. Member: Arvic Sekhon - Front end Lead



My M5 Self-Assessment

From Arric Sekhon <asekhon@sfsu.edu>

Date Wed 17/12/2025 09:54

To Amer Music <amusic@mail.sfsu.edu>; Derek Ye <dye3@sfsu.edu>; Rishika Bharodiya <rbharodiya@sfsu.edu>; Daniel Leonardo Castillo <dcastillo8@sfsu.edu>; Alonzo Jacob Manosca <amanosca1@sfsu.edu>; Han-Sheng Wang <hwang26@sfsu.edu>

Hi team,
Here is my self-assessment for Milestone 5.

1) My Contributions (Front-end/UI Lead)

- Built and improved many front-end pages (tutor profile, home, results, dashboard, mobile layouts, navigation, tutor cards).
- Added meeting features like creating meeting links, showing the join button at the right time, and fixing time zone issues.
- Cleaned up project settings (server setup, .gitignore, database files) and helped with milestone documents.
- Reviewed pull requests, helped merge branches, and made sure the UI looked consistent.

2) GitHub Commits

I made 35 **commits** to GitHub repo.

3) Main Challenges

Handling time zones for sessions was confusing at first.
Making the website look good on both desktop and mobile took several rounds of fixes.
Merging different UI branches sometimes caused small bugs I had to fix.
Updating wording and branding while keeping the layout clean also took extra time.

4) Experience with GenAI

I used GenAI to help write HTML/CSS, understand layout problems, and explore ideas for meeting links.
It was helpful for quick drafts, but I always had to double-check the results.

5) What I'd Do Differently Next Time

I would set UI design rules earlier, make smaller PRs, and use tools to catch layout problems sooner.
I'd also plan more time for time-zone issues and be clearer about who owns each UI part.

6) Anything Else

My commit count is high because I did both feature work and cleanup work across different branches.
Some of my testing and coordination isn't shown in GitHub but helped the project run smoothly.

Let me know if you need anything.

Thanks!
Arric Sekhon

2. Member: Daniel Castillo - Front end Support

Re: My M5 Self-Assessment

Daniel Leonardo Castillo

To: Alonzo Jacob Manosca; Aric Sekhon; Amer Music; Derek Ye; Rishika Bharodiya; Han-Sheng Wang

Tue 12/16/2025 10:28 PM

Hey team,

Here is my self-assessment for Milestone 5.

1) My Contributions (Front-End Support / Documentation Support)

- Revised and cleaned up front-end code by removing dead or unused code toward the end of development.
- Implemented UI and usability improvements based on feedback received from outside references and reviews.
- Built the Student Profile HTML page ensuring consistency with the rest of the site's layout and design.
- Assisted with filling in remaining tasks as needed across the milestone, helping wherever gaps existed.
- Served as the scribe for the Milestone 3 meeting and completed the M3 documentation since I had the notes.
- Focused on overall usability, reviewing pages for clarity, consistency, and ease of use.

2) GitHub Commits

I made approximately 10-11 commits to the team Dev branch. My commit count is lower than some teammates because my contributions were split between code revisions, documentation, usability review, and support tasks that are not always reflected directly in commits much like the same can be said for my other teammates.

3) Main Challenges

One challenge was trying to identify ways to contribute on the backend; while I explored this, I ultimately focused on front-end and documentation work where I could be most effective. Balancing project responsibilities alongside personal challenges related to my mother's immigration case also required careful time management. Additionally, tracking and resolving minor UI bugs required some focus.

4) Experience with GenAI

I used GenAI primarily for debugging assistance, reasoning through layout behavior, and brainstorming page ideas. It was especially helpful for understanding front-end issues more quickly, but I always verified suggestions manually. Nonetheless I found it pretty useful.

5) What I'd Do Differently Next Time

Next time, I would aim to define my technical focus earlier in the project to expand my contributions more deeply in one area. I would also communicate availability and task ownership more explicitly early on and allocate time to learn unfamiliar backend components sooner to help out in any area.

6) Anything Else

Nothing much else other than my commit count does not fully reflect my overall contribution. We all had balanced tasks and a lot of my work was split between documentation, usability review, meeting support, and assisting teammates as needed to keep the project moving smoothly.

Thanks,
Daniel Castillo

[Get Outlook for iOS](#)

3. Member: Stanley Wang - Back-end Support



Han-Sheng Wang



收件者: Derek Ye; Arrie Sekhon; Rishika Bharodiya; Daniel Leonardo Castillo; Alonzo Jacob Manosca; Amer Music

Hi team,
Here is my self-assessment for Milestone 5.

1) My Contributions (Back-end/Support)

Stanley Wang:

- Built backend data models and sample data used for system-wide testing
- Performed backend debugging, identified failures in API routes, and fixed logic errors
- Testing and fix the bug in the backend
 - Message
 - Edit Student Profile
 - Admin
- Provided continuous backend support to the Backend Lead to maintain consistency across routes and database schemas

2) GitHub Commits

I made 11 commits to GitHub repo.

3) Main Challenges

Handling user authentication and data validation correctly required careful design and testing.
Managing database schema changes across branches sometimes caused merge conflicts that needed manual fixes.
Debugging backend logic was time-consuming, especially when issues only appeared with specific inputs or edge cases.

4) Experience with GenAI

I used GenAI to help draft backend code, review logic, and think through edge cases for features like registration and data handling.
It was useful for speeding up development and clarifying ideas, but I always reviewed and tested the code to ensure it met the project requirements.

5) What I'd Do Differently Next Time

I would define API contracts and data models earlier to reduce rework later.
I would also make smaller, more focused pull requests and add automated tests sooner to catch issues earlier.
In addition, I'd clarify backend ownership and responsibilities earlier to improve coordination.

6) Anything Else

My commit count may look a bit low because much of my work involved backend debugging, testing, and coordination that is not fully reflected in GitHub.

Let me know if you need anything.

Thanks!

Stanley Wang

4. Member: Derek Ye - Back-end Lead



Derek Ye



To: Arrie Sekhon; Rishika Bharodiya; Daniel Leonardo Castillo; Han-Sheng Wang; Alonzo Jacob Manosca; Amer Music

Hello! This is Derek. Here is my self assessment.

a) Their contributions to team project and teamwork (technical and any other) in no more than half a page – list item format is OK. List also the role (e.g. front-end lead).

I was the Backend lead.

Here are my contributions to this project:

- Creation of most tables in our SQL server
- Connection of front end features to backend logic. Specifically:
 - The admin dashboard with application approval / denial
 - Tutor search
 - Session history on the student and tutor dashboards
- Some minor front end features:
 - Admin dashboard before it was changed
 - Some minor front end issues that needed to be fixed. We had a to do list, but some things include the size of certain HTML elements being disproportional or misaligned with other page content.
- I also pasted 4 lines to add Google Analytics to our project.

b) Number of submissions they made to GitHub team Dev. Branch (explain if this number is very low)

I made 17 commits to our development branch. I think this number might be a little low because I don't usually commit to Github until I fully finish + test out a feature.

c) One brief paragraph on the main challenges they encountered in team project

My main challenge was trying to adapt to new changes to our website. Sometimes my teammates would push lots of new frontend changes or changes to the server (which is NOT a bad thing) and it would take me a little bit to adjust to their new additions + think about how to create backend logic to accompany it.

d) One brief paragraph of their own overall experience with GenAI focusing on tasks it was applied to and how useful it was

I used GenAI to debug as well as create new features in the backend. I'm not too experienced with developing in Python yet so it was very helpful when I was feeling lost. Overall, I think GenAI was incredibly useful and saved me many hours of debugging!

e) One brief paragraph on what would do better next time based on what was learned in the class about SE management and processes f) Anything else you deem important for instructors to know (e.g. why your GitHub count is low)

One thing I would do better is keep myself updated better on our website. I shifted my focus for a little bit to my other classes while a lot of changes were being made to this project and it was hard to catch up, but eventually I was able to. Overall I am really happy I got such amazing teammates, this project was a great experience!

Reply Reply all Forward ...
Tue 12/16/2025 10:17 PM

5. Member : Rishika Bharodiya - Github Master

My M5 self assessment - Team 01

Summarize

RB

Rishika Bharodiya

To: Aric Sekhon; Daniel Leonardo Castillo; Han-Sheng Wang; Alonzo Jacob Manosca; Amer Music

Tue 12/16/2025 11:03 PM

😊

↩ Reply

↩ Reply all

➡ Forward

🔍

📧

⋮

Hi Team,

1. Contributions on backend :

- Implementation of the full 3-role user system (Student, Tutor, Admin) with role-based access and permissions.
- Created the Tutor Application & Approval flow:
 - Application form
 - Pending queue
 - Admin review page
 - Automatic student → tutor promotion
- contributed in Building the Two-Sided Dashboard:
 - Student view (bookings + statuses)
 - Tutor view (incoming requests + confirm/reject), the Booking System allowing students to request sessions with tutors.
- Added the Messaging System for direct student-tutor communication.
- contributed in Implemented backend logic for user roles, applications, tutor profiles, bookings, and messages.
- Ensured dynamic routing so the website changes based on the logged-in user's role.
- . implemented jisthi code
- helped in documentation .

2. GitHub Commits:

I made 4 commits on the main branch.

3) Main Challenges

- * Facing laptop issues that prevented me from pushing code directly to GitHub.
- * Balancing quick prototyping with ensuring code correctness and functionality.
- * Coordinating with the team to ensure my local work was integrated properly.

4) Experience with GenAI

I used GenAI to help understand and polish code, as well as debug errors. It was helpful for quick drafts, but I always double-checked the results to ensure accuracy.

5) What I'd Do Differently Next Time

I would like to spend more time working on the front-end side, as it really interests me.

6) Anything Else

My commit count is lower because I was facing issues with my laptop and was unable to push to GitHub. I worked locally and shared my code with the team via Discord and email so they could push it on my behalf.

Thanks,
Rishika Bharodiya

6. Member: Alonzo Manosca - Github Support

Re: My M5 Self-Assessment

Alonzo Jacob Manosca

To: Arric Sekhon; Derek Ye; Rishika Bharodiya; Daniel Leonardo Castillo; Han-Sheng Wang

Tue 12/16/2025 10:17 PM

Hey guys, here is my self assessment for milestone 5.

1) My Contributions

- Assisted in managing Github, monitoring project organization and ensuring smooth commits for all changes.
- Worked on and assisted others in working on project documentation.
- Cleaned up and checked documents for potential spelling, grammar, or miscellaneous errors.

2) GitHub Commits

I made 3 commits to the team's Github repository.

3) Main Challenges

One of the main challenges in working on a project such as this was coordination. I had to work with my other teammates to distribute tasks and divide the work, which proved to be an insightful learning experience. Another challenge in working on this project was getting comfortable with the different tools we utilized. Getting accustomed to the team's workflow was difficult at first, but became much more comfortable over time.

4) Experience with GenAI

I tried to not use AI too heavily, and as such, only used it in a very limited capacity. I only used it when I found myself stuck or needed support on a task.

5) What I'd Do Differently Next Time

Next time, I would try and offer more support to the front and back-end teams in order to expedite our process and potentially speed up the rate at which we did work. I would also try and work with my teammates to organize our project information a little better as sometimes certain information such as passwords were quickly buried in our discord chats.

6) Anything Else

My commits are much lower than my other teams as I focused more on technical tasks, such as working on documentation or reviewing work.

Thanks,

Alonzo Manosca

7. Member: Amer Music - Project Lead

Amer Music

To: Han-Sheng Wang; Derek Ye; Arric Sekhon; Rishika Bharodiya; Daniel Leonardo Castillo; Alonzo Jacob Manosca

Hi team,
Here is my self-assessment for Milestone 5.

1) My Contributions

Amer Music:

- Assigned and coordinated project tasks to team members.
- Created the initial front end & back end for the following pages:
 - Home
 - About
 - Member
 - Search
 - Results
 - Tutor Dashboard
 - Edit My Profile
 - Manage Availability
 - Become a Tutor
 - Base
 - 404
 - 500
 - Style CSS
 - App.py
- Assisted in refining front end through functional Figma Mockups.

2) GitHub Commits

I made 39 commits to GitHub repo.

3) Main Challenges

The main challenges I encountered were centered around adjusting existing code as functionalities and new pages were being developed.

4) Experience with GenAI

I used GenAI to draft visual mock-ups (Figma), de-bugging code, and suggesting code for certain functionalities.

5) What I'd Do Differently Next Time

Agreeing on the pages and functionalities of the web app in the beginning of the project may have led to a smoother development and a more efficient code overall.

6) Anything Else

The team experience has been exceptionally smooth in terms of coordination and team effort.

Let me know if you need anything.

Thanks!

Overall feedback from team lead:

Main Challenges: From the perspective of a team lead, the primary challenge that I encountered during this project was coordinating meeting times that aligned with the availability of all team members. Oftentimes, we would summarize the minutes of the meeting for those that were unavailable to attend. Otherwise, there were remarkably few items that posed any substantive challenge to the project.

Experience Using GenAI: Early on, I approached the use of GenAI with a narrow scope. Experimenting with code generation quickly proved inefficient, as the suggested code became convoluted or buggy beyond a few lines of code. Debugging worked very well in identifying dead/unused code, and often suggested effective patches as long as it was focused on a narrow issue. Using GenAI (like Figma) to generate interactive visual mockups of pages was incredibly helpful and gave the team a shared direction for each page.

What I would do better: Based on what I've learned throughout this course and as a team lead for a software engineering project, the one change I would have made would have been to assign and coordinate tasks on platforms like clickup much sooner. The interface allowed each member to see deadlines and tasks/subtasks they were assigned to for each milestone, as well as a description of the task and the respective deliverable. Employing this tool earlier in the process, in addition to a more comprehensive outline of what the final project would contain, would have made certain milestones more efficient.